

```

import java.util.*;
import java.util.stream.*;

class Student {

    int id;
    String name;
    List<String> courses;
    Map<String, Integer> scores;

    public Student(int id, String name, List<String> courses, Map<String,
Integer> scores) {
        this.id = id;
        this.name = name;
        this.courses = courses;
        this.scores = scores;
    }

    // Calculate average score
    public double getAverageScore() {
        int sum = scores.values().stream().mapToInt(Integer::intValue).sum();
        return scores.size() == 0 ? 0 : (double) sum / scores.size();
    }

    // Handle missing course using getOrDefault
    public int getScore(String course) {
        return scores.getOrDefault(course, 0);
    }
}

public class StudentPerformanceAnalyzer {

    // 1. Get Top N Students based on average score
    public static List<Student> getTopNStudents(List<Student> students, int n) {

        return students.stream()

.sorted(Comparator.comparingDouble(Student::getAverageScore).reversed())
        .limit(n)
        .collect(Collectors.toList());
    }

    // 2. Get average score per course
    public static Map<String, Double> getAverageScorePerCourse(List<Student>
students) {

```

```

Map<String, Double> courseAverage = new HashMap<>();
Set<String> allCourses = getAllUniqueCourses(students);

for (String course : allCourses) {

    double avg = students.stream()
        .mapToInt(s -> s.getScore(course))
        .average()
        .orElse(0);

    courseAverage.put(course, avg);
}

return courseAverage;
}

// 3. Get all unique courses
public static Set<String> getAllUniqueCourses(List<Student> students) {

    Set<String> courses = new HashSet<>();

    for (Student s : students) {
        courses.addAll(s.courses);
    }

    return courses;
}

public static void main(String[] args) {

    List<Student> students = new ArrayList<>();

    Map<String, Integer> scores1 = new HashMap<>();
    scores1.put("Math", 85);
    scores1.put("Physics", 90);

    Map<String, Integer> scores2 = new HashMap<>();
    scores2.put("Math", 75);
    scores2.put("Physics", 80);
    scores2.put("Chemistry", 70);

    Map<String, Integer> scores3 = new HashMap<>();
    scores3.put("Math", 95);
    scores3.put("Physics", 88);
}

```

```

students.add(new Student(1, "Aman",
    Arrays.asList("Math", "Physics"), scores1));

students.add(new Student(2, "Rahul",
    Arrays.asList("Math", "Physics", "Chemistry"), scores2));

students.add(new Student(3, "Priya",
    Arrays.asList("Math", "Physics"), scores3));

// Top 2 students
List<Student> topStudents = getTopNStudents(students, 2);

System.out.println("Top Students:");
for (Student s : topStudents) {
    System.out.println(s.name + " Average Score: " +
s.getAverageScore());
}

// Average per course
Map<String, Double> courseAvg = getAverageScorePerCourse(students);
System.out.println("\nAverage Score Per Course:");
System.out.println(courseAvg);

// Unique courses
Set<String> uniqueCourses = getAllUniqueCourses(students);
System.out.println("\nUnique Courses:");
System.out.println(uniqueCourses);
}
}

```

OUTPUT

Top Students:

Priya Average Score: 91.5

Aman Average Score: 87.5

Average Score Per Course:

{Chemistry=23.33333333333332, Math=85.0, Physics=86.0}

Unique Courses:

[Chemistry, Math, Physics]

Time Complexity Analysis

1. Time Complexity of Computing Course Averages

Let:

n = number of students

m = number of unique courses

First, all unique courses are collected from the list of students. Each student may have multiple courses, so this step takes $O(n \times m)$ time in the worst case.

Next, for each course, the program iterates through all students to obtain their scores using `getOrDefault()` and computes the average using streams. This requires $O(n)$ time for each course.

Since this process is repeated for m courses, the total time complexity becomes:

$$O(nxm)$$

Therefore, the time complexity of computing the average score per course is $O(nm)$.

2. Time Complexity of Sorting Top N Students

To obtain the top N students, the program sorts the list of students according to their average scores using a Comparator.

Sorting n students requires:

$$O(n \log n)$$

After sorting, the stream operation `limit(n)` selects the first N students, which takes $O(N)$ time.

Since

$$O(N)$$

$O(N)$ is smaller than $O(n \log n)$, the overall time complexity is dominated by the sorting step.

Therefore, the complexity of finding the top N students is:

$O(n \log n)$